



Московский государственный университет имени М.В. Ломоносова

Факультет космических исследований

Жижченко Александр Алексеевич,
Ракчеев Георгий Андреевич

Турнир Угольникова

Москва, 2025

Содержание

1	Общая постановка задачи	2
2	Постановка задачи с разбивкой на зоны ответственности участников	2
3	Теоретический раздел	2
3.1	Используемые модели	2
3.2	Методы решения	2
3.3	Используемые данные	3
4	Раздел с описанием кода	3
4.1	Диаграмма классов	3
4.2	Описание классов	4
4.2.1	Шаблонный класс <code>List<T></code>	4
4.2.2	Класс <code>Queue</code>	6
4.2.3	Класс <code>String</code>	7
4.2.4	Структуры <code>Coordinates</code> и <code>Polygon</code>	8
4.2.5	Класс <code>AbstractSubject</code>	8
4.2.6	Класс <code>SubjectRussia</code>	9
4.2.7	Класс <code>Map</code>	9
4.2.8	Класс <code>Game</code>	10
4.2.9	Класс <code>MapWidget</code>	10
4.2.10	Класс <code>PlayerWindow</code>	10
4.2.11	Класс <code>Presenter</code>	11
4.3	Общий алгоритм работы программы	12
4.4	Алгоритм тестирования программы	12
4.4.1	Интерфейс тестирования	12
4.4.2	Процедура тестирования	12
4.5	Недостатки программы и предложения по их исправлению	13
5	Примеры тестов ПО и обоснование корректности работы кода	13
5.1	Тест 1: Корректный ход игрока	13
5.2	Тест 2: Некорректный ход	13
6	Выводы о проделанной работе	13
7	Отзывы протестировавших сокурсников	14
8	Список литературы	14

1 Общая постановка задачи

Целью проекта является разработка интерактивной логической игры «Турнир Угольников», в которой пользователь и компьютер по очереди перемещаются по карте регионов Российской Федерации. Карта представляется в виде графа, вершинами которого являются регионы, а рёбрами — факт наличия общей границы между регионами.

Игра начинается в случайном стартовом регионе и имеет заранее выбранный конечный регион. Задача игрока — первым достичь конечного региона, перемещаясь только в соседние, ранее не посещённые регионы. Компьютер выступает в роли противника и использует алгоритмическую стратегию выбора ходов.

Проект реализован с использованием языка C++ и фреймворка Qt 6 и включает графический интерфейс, визуализацию карты, игровую логику и алгоритмы принятия решений.

2 Постановка задачи с разбивкой на зоны ответственности участников

1. Жижченко Александр Алексеевич:

- Реализация структуры данных карты и регионов
- Загрузка и парсинг данных из GeoJSON
- Визуализация карты и реализация MapWidget
- Интеграция интерфейса Qt и архитектуры Presenter

2. Ракчеев Георгий Андреевич:

- Разработка игровой логики и правил игры
- Реализация класса Game и алгоритма хода компьютера
- Реализация алгоритмов поиска (BFS) и анализа достижимости

3 Теоретический раздел

3.1 Используемые модели

В основе проекта лежит модель неориентированного графа:

- вершины графа — регионы (субъекты РФ);
- рёбра — наличие общей границы между регионами.

Каждая вершина может находиться в состоянии «посещена» или «не посещена», что влияет на допустимость ходов. Игра представляет собой пошаговую игру двух игроков с полной информацией.

3.2 Методы решения

Для реализации стратегии компьютера используется:

- поиск в ширину (BFS) от конечного региона для вычисления расстояний;
- анализ чётности расстояний до конечного региона;

- выбор оптимального хода на основе минимального расстояния и чётности.

Такой подход позволяет компьютеру играть стратегически корректно и избегать заведомо проигрышных позиций.

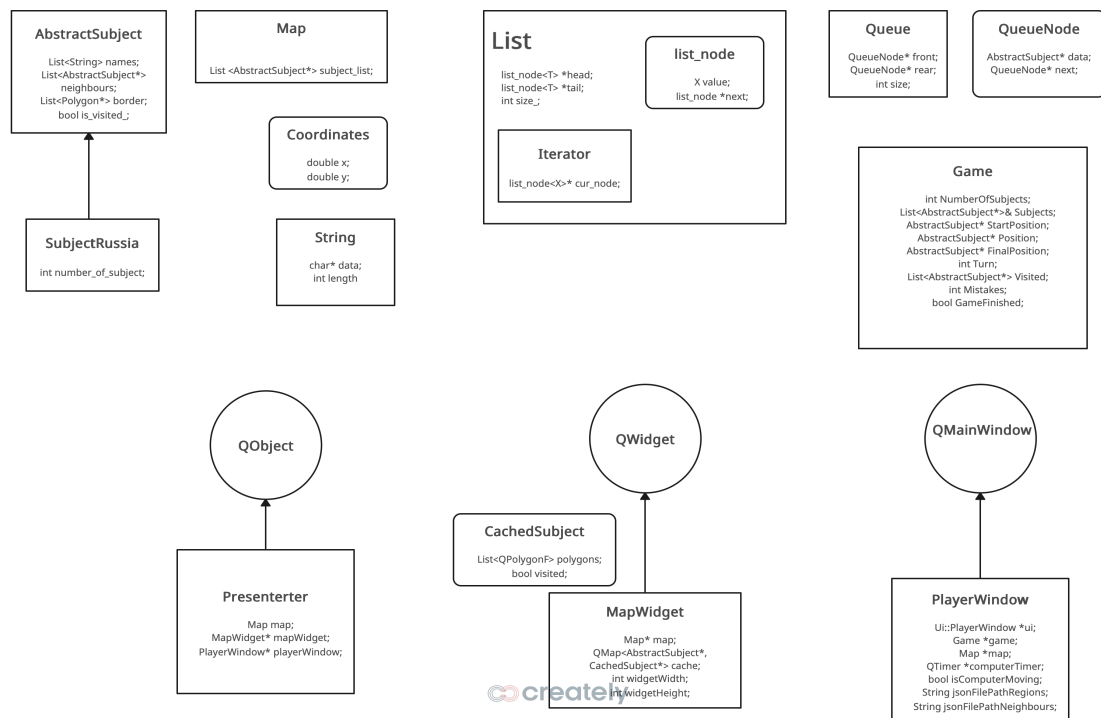
3.3 Используемые данные

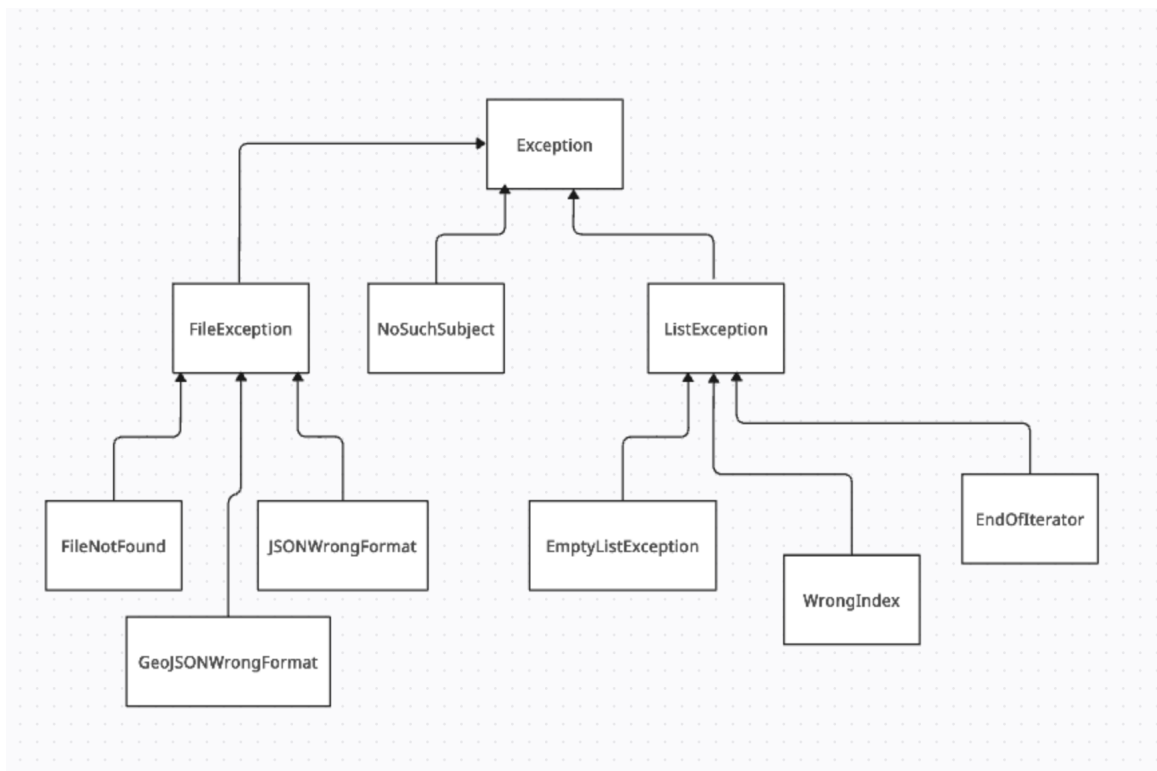
Исходные данные включают:

- файл GeoJSON, содержащий геометрию регионов РФ;
- файл со списком соседних регионов;
- внутренние структуры данных List, Queue.

4 Раздел с описанием кода

4.1 Диаграмма классов





Основные классы проекта:

- Game
- AbstractSubject
- Map
- MapWidget
- PlayerWindow
- Presenter

4.2 Описание классов

4.2.1 Шаблонный класс `List<T>`

Шаблонный класс `List<T>` реализует собственную односвязную динамическую структуру данных — список. Данный класс используется во всём проекте как базовый контейнер для хранения регионов, строк, координат и других объектов, что позволяет избежать зависимости от стандартной библиотеки STL и полностью контролировать поведение контейнера.

Внутренняя структура:

- `list_node<T>` — вложенная структура узла списка, содержащая:
 - `value: T` — хранимое значение.
 - `next`: указатель на следующий узел списка.

Поля класса:

- `list_node<T>* head` — указатель на первый элемент списка.
- `list_node<T>* tail` — указатель на последний элемент списка.

- `int size_` — текущее количество элементов в списке.

Конструкторы и деструктор:

- `List()` — конструктор по умолчанию, создаёт пустой список.
- `List(const List&)` — копирующий конструктор удалён для предотвращения неявного копирования.
- `List(List&& other)` — перемещающий конструктор, передающий владение данными.
- `~List()` — деструктор, освобождающий всю динамически выделенную память.

Основные методы работы со списком:

- `void push(T value)` — добавляет элемент в начало списка.
- `void Add(T value)` — добавляет элемент в конец списка.
- `T pop()` — удаляет и возвращает первый элемент списка; при попытке извлечения из пустого списка выбрасывает исключение `EmptyListError`.
- `void clear()` — полностью очищает список.
- `int size() const` — возвращает текущее количество элементов.

Доступ к элементам:

- `T& Get(int index)` — возвращает ссылку на элемент с заданным индексом.
- `const T& Get(int index) const` — константная версия метода доступа.

При выходе индекса за допустимые границы генерируется исключение `Error`.

Итератор:

- Вложенный шаблонный класс `Iterator<X>` реализует последовательный обход списка.
- Метод `next()` возвращает текущий элемент и сдвигает итератор вперёд; при выходе за пределы списка выбрасывается исключение `EndOfIterator`.
- Метод `isEnd()` проверяет завершение обхода.
- Метод `iter()` возвращает итератор для текущего списка.

Поисковые методы:

- `bool in_list(T comp_obj)` — проверяет, содержится ли объект в списке.
- `T* find_in_list(T comp_obj)` — возвращает указатель на найденный объект или `nullptr`, если элемент отсутствует.

Роль класса в проекте: Класс `List<T>` используется как универсальный контейнер для:

- хранения регионов карты;
- представления графа смежности;
- хранения имён регионов;
- хранения геометрических данных (координат и многоугольников).

Использование собственного контейнера позволило упростить контроль памяти, внедрить собственную систему обработки ошибок и обеспечить единообразие работы со структурами данных во всех модулях программы.

4.2.2 Класс Queue

Класс `Queue` реализует очередь (FIFO — First In, First Out) для хранения указателей на объекты типа `AbstractSubject`. Очередь используется в алгоритмах обхода графа карты, в частности при реализации поиска в ширину (BFS) для вычисления расстояний между регионами в игровом процессе.

Вспомогательная структура:

- `QueueNode` — узел односвязного списка, представляющего очередь.
 - `AbstractSubject* data` — указатель на регион карты.
 - `QueueNode* next` — указатель на следующий элемент очереди.

Поля класса:

- `QueueNode* front` — указатель на первый элемент очереди.
- `QueueNode* rear` — указатель на последний элемент очереди.
- `int size` — текущее количество элементов в очереди.

Конструктор и деструктор:

- `Queue()` — конструктор, инициализирующий пустую очередь.
- `~Queue()` — деструктор, освобождающий всю динамически выделенную память путём очистки очереди.

Основные методы:

- `void add(AbstractSubject* data)` — добавляет элемент в конец очереди.
 - `data` — указатель на объект региона, помещаемый в очередь.
- `AbstractSubject* remove()` — удаляет и возвращает элемент из начала очереди.
 - Используется при последовательном извлечении элементов в алгоритме BFS.
- `AbstractSubject* getFront() const` — возвращает элемент из начала очереди без его удаления.

Методы состояния:

- `bool isEmpty() const` — проверяет, пуста ли очередь.
- `int getSize() const` — возвращает количество элементов в очереди.
- `void clear()` — полностью очищает очередь, освобождая память всех узлов.

Роль класса в проекте: Класс `Queue` используется исключительно во внутренних алгоритмах игры, а именно:

- при обходе графа регионов карты;
- для вычисления кратчайших расстояний от текущей позиции до целевого региона;
- в логике принятия решений компьютером (выбор оптимального хода).

Реализация очереди в виде односвязного списка позволяет эффективно выполнять операции добавления и удаления за константное время $O(1)$, что критично для работы алгоритмов обхода графа.

4.2.3 Класс String

Класс `String` реализует собственную строковую структуру данных и используется в проекте вместо стандартного класса `std::string`. Основной целью введения данного класса является полный контроль над управлением памятью, а также единый стиль работы со строками во всех пользовательских структурах данных.

Поля класса:

- `char* data` — динамически выделенный массив символов, содержащий строку в формате C-style (с завершающим нулевым символом).
- `int length` — длина строки без учёта завершающего нулевого символа.

Вспомогательные методы:

- `void copy_from(const char* str)` — копирует C-строку в текущий объект, выделяя новую память.
- `void copy_from(const String& other)` — копирует содержимое другого объекта `String`.

Данные методы инкапсулируют логику копирования и используются конструкторами и операторами присваивания.

Конструкторы и деструктор:

- `String()` — конструктор по умолчанию, создаёт пустую строку.
- `String(const char* cstr)` — создаёт объект строки из C-строки.
- `String(const String& other)` — копирующий конструктор, выполняющий глубокое копирование данных.
- `~String()` — деструктор, освобождающий динамически выделенную память.

Операторы присваивания:

- `String& operator=(const String& other)` — копирующее присваивание с защитой от самоприсваивания.
- `String& operator=(const char* cstr)` — присваивание из C-строки.
- `String& operator=(String&& other)` — перемещающее присваивание, передающее владение памятью без копирования.

Методы доступа и состояния:

- `int size() const` — возвращает длину строки.
- `bool empty() const` — проверяет, является ли строка пустой.
- `const char* c_str() const` — возвращает указатель на C-строку.

Доступ к символам:

- `char& operator[](int index)` — доступ к символу строки по индексу.
- `const char& operator[](int index) const` — константная версия доступа.

Индексация используется преимущественно во внутренних алгоритмах обработки строк.

Операторы сравнения:

- `bool operator==(const String& other) const` — проверка строк на равенство.
- `bool operator!=(const String& other) const` — проверка строк на неравенство.

Роль класса в проекте: Класс `String` используется:

- для хранения имён регионов;
- при разборе JSON-файлов;
- в логике игрового процесса (ввод игрока, сравнение названий регионов);
- в пользовательских структурах данных (`List<String>`).

Использование собственного класса строк позволило обеспечить совместимость с остальными самописными контейнерами, упростить отладку и избежать скрытых накладных расходов стандартной библиотеки.

4.2.4 Структуры `Coordinates` и `Polygon`

Для описания геометрии регионов используются вспомогательные структуры данных.

Структура `Coordinates`:

- `x: double` — координата по оси X.
- `y: double` — координата по оси Y.

Данная структура используется для задания вершин многоугольников, описывающих границы регионов.

Тип `Polygon`:

- `Polygon` — псевдоним для `List<Coordinates>`, представляющий собой упорядоченный список вершин одного многоугольника.

4.2.5 Класс `AbstractSubject`

Класс `AbstractSubject` является базовым абстрактным представлением региона (субъекта) на карте. Он объединяет логическую информацию, используемую в игре, и геометрические данные, необходимые для визуализации.

Поля:

- `List<String> names` — список названий региона; используется для поддержки альтернативных имён и пользовательского ввода.
- `List<AbstractSubject*> neighbours` — список указателей на соседние регионы, формирующий граф карты.
- `List<Polygon*> border` — набор многоугольников, описывающих границы региона на карте.
- `bool is_visited_` — флаг, показывающий, был ли регион посещён в текущей игровой сессии.

Методы:

- `add_neighbour(AbstractSubject*)` — добавляет соседний регион.

- `add_name(String)` — добавляет альтернативное название региона.
- `add_coord(Coordinates)` — добавляет вершину в текущий многоугольник границы.
- `add_polygon()` — завершает формирование текущего многоугольника и добавляет его в список границ.
- `visit()` — помечает регион как посещённый.
- `unvisit()` — сбрасывает флаг посещения региона.
- `get_names()` — возвращает список названий региона.
- `get_neighbours()` — возвращает список соседних регионов.
- `get_border()` — возвращает геометрию границ региона.
- `is_visited()` — возвращает текущее состояние флага посещения.

Класс `AbstractSubject` используется как базовый тип для всех регионов карты и обеспечивает единый интерфейс работы с ними.

4.2.6 Класс `SubjectRussia`

Класс `SubjectRussia` является конкретной реализацией региона и наследуется от `AbstractSubject`. Он предназначен для представления субъектов Российской Федерации.

Поля:

- `int number_of_subject` — числовой идентификатор субъекта Российской Федерации.

В текущей версии проекта класс расширяет базовую функциональность `AbstractSubject` и может быть использован для дальнейшего добавления специфических атрибутов, характерных именно для субъектов РФ.

4.2.7 Класс `Map`

Класс `Map` отвечает за хранение и инициализацию карты игры. Он управляет набором регионов и их связями, а также загружает данные из внешних источников.

Поля:

- `List<AbstractSubject*> subject_list` — список всех регионов, присутствующих на карте.

Методы:

- `get_from_JSON(String, String)` — загружает описание регионов и их соседства из JSON-файлов, формируя список субъектов и граф смежности.
- `is_neighbours(String, String)` — проверяет, являются ли два региона соседними по их названиям.
- `get_subjects()` — возвращает список всех регионов карты.

Вспомогательные методы:

- `find_subject_by_name(List<AbstractSubject*>&, const String&)` — статический метод поиска региона по имени в заданном списке.

Класс `Map` инкапсулирует структуру графа регионов и служит связующим звеном между игровым движком и пользовательским интерфейсом.

4.2.8 Класс Game

Поля:

- `List<AbstractSubject*> Subjects` — список всех регионов
- `AbstractSubject* Position` — текущий регион
- `AbstractSubject* FinalPosition` — конечный регион
- `List<AbstractSubject*> Visited` — список посещённых регионов
- `int Mistakes` — количество ошибок игрока

Методы:

- `int makePlayerMove(String destination)` — обработка хода игрока
- `int makeComputerMove()` — алгоритм хода компьютера
- `int* calculateDistances()` — BFS от конечного региона
- `bool isGameFinished()` — проверка завершения игры

4.2.9 Класс MapWidget

Назначение: визуализация карты регионов.

Поля:

- `Map* map` — ссылка на карту
- `QMap<AbstractSubject*, CachedSubject*> cache` — кэш геометрии

Методы:

- `paintEvent` — отрисовка карты
- `rebuildCache` — перестроение кэша при изменении состояния регионов

4.2.10 Класс PlayerWindow

Класс `PlayerWindow` отвечает за графический интерфейс игрока и реализует основное окно приложения. Он обеспечивает взаимодействие пользователя с игровой логикой, отображение текущего состояния игры и обработку ходов как игрока, так и компьютера.

Поля:

- `Ui::PlayerWindow *ui` — автоматически сгенерированный Qt-интерфейс окна, содержащий элементы управления (кнопки, поля ввода, надписи).
- `Game *game` — указатель на объект игровой логики, через который выполняются ходы и проверяется состояние игры.
- `Map *map` — указатель на карту регионов, используемую для инициализации игры и отображения данных.
- `QTimer *computerTimer` — таймер, обеспечивающий задержку между ходом игрока и ходом компьютера для улучшения восприятия игры.

- `bool isComputerMoving` — флаг, предотвращающий одновременное выполнение ходов игрока и компьютера.
- `String jsonFilePathRegions` — путь к файлу с описанием регионов.
- `String jsonFilePathNeighbours` — путь к файлу с описанием соседства регионов.

Методы:

- `PlayerWindow(Map* map, QWidget* parent)` — конструктор, инициализирующий окно, карту и элементы интерфейса.
- `void addMap(Map* m)` — установка или замена карты регионов.
- `void initGame()` — инициализация новой игры, создание объекта `Game` и начальной позиции.
- `void updateUI()` — обновление элементов интерфейса в соответствии с текущим состоянием игры.
- `void on_makeMoveButton_clicked()` — обработка нажатия кнопки хода игрока.
- `void on_regionInput_returnPressed()` — обработка ввода региона через текстовое поле.
- `void onComputerMove()` — выполнение хода компьютера по таймеру.
- `void handleGameResult(int result)` — обработка результатов хода (победа, поражение, ошибка).
- `QStringList getAllRegionNames() const` — получение списка всех доступных регионов для автодополнения и проверки ввода.
- `void processPlayerMove(const QString& regionName)` — проверка и выполнение хода игрока.

Сигналы:

- `regionVisited(const QString& regionName)` — испускается при успешном посещении нового региона и используется `Presenter`'ом для обновления карты.
- `gameFinished()` — сигнал завершения игры, используемый для очистки кэша и завершения текущей игровой сессии.

4.2.11 Класс `Presenter`

Реализует взаимодействие всех элементов программы. **Поля:**

- `Map map` — объект карты, содержащий описание регионов и их соседства; используется одновременно игровой логикой и виджетом карты.
- `MapWidget *mapWidget` — указатель на виджет визуализации карты, отвечающий за отрисовку регионов и их состояния.
- `PlayerWindow *playerWindow` — указатель на игровое окно пользователя, обеспечивающее ввод ходов и отображение текстовой информации.

Методы:

- `Presenter(QObject* parent)` — конструктор, инициализирующий карту, игровое окно и виджет карты, а также настраивающий сигнально-слотовые соединения между ними.
- `~Presenter()` — деструктор, освобождающий динамически выделенные ресурсы.
- `MapWidget* getMapWidget() const` — возвращает указатель на объект `MapWidget` для встраивания в главное окно приложения.
- `PlayerWindow* getPlayerWindow() const` — возвращает указатель на объект `PlayerWindow` для отображения игрового интерфейса.

Слоты:

- `onRegionVisited(const QString& regionName)` — вызывается при сигнале о посещении региона игроком; обновляет состояние соответствующего субъекта на карте и инициирует перерисовку `MapWidget`.
- `onGameFinished()` — вызывается по завершении игры; очищает кэш субъектов карты и подготавливает систему к возможному запуску новой игровой сессии.

Класс `Presenter` не содержит логики пользовательского интерфейса или игровой механики напрямую, но обеспечивает их согласованную работу, минимизируя связность между компонентами и упрощая расширение и сопровождение проекта.

4.3 Общий алгоритм работы программы

1. Загрузка карты и данных о регионах
2. Случайный выбор стартового и конечного регионов
3. Ход игрока через интерфейс
4. Проверка корректности хода
5. Ход компьютера
6. Обновление карты и интерфейса
7. Завершение игры при достижении конечного региона или ошибках

4.4 Алгоритм тестирования программы

4.4.1 Интерфейс тестирования

Тестирование осуществляется через графический интерфейс пользователя с визуальным контролем состояния карты.

4.4.2 Процедура тестирования

1. Запуск приложения
2. Выполнение корректных и некорректных ходов
3. Проверка реакции интерфейса и карты
4. Проверка завершения игры

4.5 Недостатки программы и предложения по их исправлению

- **Недостаток:** отсутствие сохранения состояния игры
 - **Исправление:** добавить сериализацию состояния
- **Недостаток:** фиксированная стратегия компьютера
 - **Исправление:** реализация нескольких уровней сложности
- **Недостаток:** Возможность игры только на карте России
 - **Исправление:** Добавление новых стран и карты звёздного неба.
- **Недостаток:** Ошибки в подсчётах жизней игрока
 - **Исправление:** Изменение принципа снятия жизни (возможно, это стоит делать в другом месте)

5 Примеры тестов ПО и обоснование корректности работы кода

5.1 Тест 1: Корректный ход игрока

- **Исходные данные:** соседний регион
- **Ожидаемый результат:** переход выполнен
- **Полученный результат:** переход выполнен
- **Вывод:** логика хода работает корректно

5.2 Тест 2: Некорректный ход

- **Исходные данные:** несоседний регион
- **Ожидаемый результат:** ошибка
- **Полученный результат:** ошибка зафиксирована
- **Вывод:** проверка корректности реализована правильно

6 Выводы о проделанной работе

В ходе работы была разработана полнофункциональная логическая игра с визуализацией карты, реализованы алгоритмы поиска и стратегия противника, а также применены современные средства разработки на базе Qt6 и C++.

7 Отзывы протестировавших сокурсников

- Сокурсник 1: Игра понравилась, хотя и была слишком сложной, так как за всё время обучения в школе я был на географии 3 раза, а за пределами Москвы – всего 2 (Дубик Дмитрий)
- Сокурсник 2: Игра понравилась, оказалось, что я немного знаю регионы и даже смог обыграть компьютер один раз. В работе программы присутствуют недочёты, но было увлекательно. (Анохин Серафим)
- Сокурсник 3: Мне понравилось, думаю, игру можно будет использовать на уроках географии.

8 Список литературы

Список литературы

- [1] Qt Documentation. <https://doc.qt.io>
- [2] Т. Кормен и др. Алгоритмы. Построение и анализ.
- [3] GeoJSON Specification. <https://geojson.org>
- [4] Примеры работы с QT <https://metanit.com>